



UNIVERSITY OF CAPE TOWN

DEPARTMENT OF MATHEMATICS AND APPLIED
MATHEMATICS

MASTER OF SCIENCE IN MATHEMATICS

Disentangling Sequential Goal Representations in the Procgen-Heist Environment

Author:
Benjamin Sturgeon

Student Number:
STRBEN005

May 2025

Supervisor:
Jonathan Shock

A thesis submitted in fulfillment of the requirements

Disentangling Sequential Goal Representations in the Procgen-Heist
Environment Benjamin Sturgeon

for the degree of Master of Philosophy

Declaration

I, Benjamin Sturgeon, hereby declare that the work on which this dissertation/thesis is based is my original work (except where acknowledgements indicate otherwise) and that neither the whole work nor any part of it has been, is being, or is to be submitted for another degree in this or any other university.

I empower the university to reproduce for the purpose of research either the whole or any portion of the contents in any manner whatsoever.

Signature

Date

Abstract

Sparse AutoEncoders (SAEs) have proven to be useful tools in understanding Large Language Models as a means of separating out features within neural networks. In this work we explore the efficacy of this tool in understanding a maze solving reinforcement learning agent trained in the Procgen Heist environment. We target this model because it initially presented a high degree of polysemanticity due to the fact that it has to target multiple different entities to succeed. Our first finding was that there are many channels that when manipulated in isolation allow for complete re-targeting of the agent. We discovered that a vast majority of the "steering channels" in the SAE were in fact dead channels that didn't activate at all during normal operation. We further found that the model was capable of steering itself to different entities in the maze, even when every other channel in the layer was completely ablated, and that there were significantly fewer channels capable of this in our SAEs. Finally, we discovered that the SAEs were in fact not splitting entities across multiple channels, but were in fact multiplexing entity information within the same channel and through varying the activation strength, sending a signal regarding the entity they were targeting. We believe these findings may provide insight into the application of SAEs more broadly, and help to shed light on the operations of reinforcement learning models.

Acknowledgments

Contents

Abstract	2
Acknowledgments	3
1 Introduction	9
1.1 Background and Motivation	10
1.2 Problem Statement	14
1.3 Research Objectives	15
1.4 Research Questions	15
1.5 Key Hypotheses	16
1.6 Scope and Limitations	17
1.7 Significance of the Study	18
1.8 Thesis Structure	18
2 Literature Review	19
2.1 Introduction	19
2.2 Reinforcement Learning Fundamentals	19
2.2.1 Markov Decision Processes	20
2.2.2 Value Functions and Policies	20
2.2.3 Deep Reinforcement Learning	20
2.3 Reinforcement Learning Safety	21
2.3.1 Safety Problems in RL	21
2.4 Interpretability in Machine Learning	23
2.4.1 Visualising neural networks	25
2.4.2 Post-hoc Explanations	27
2.4.3 Intrinsically Interpretable Models	28
2.5 Interpretability in Reinforcement Learning	29
2.6 The Procgen Suite of environments	31
2.7 Understanding and Controlling a Maze Solving Policy Network	32
2.7.1 Discovering and Controlling Goal Representations . . .	33

2.8	Gaps in the Literature	33
2.9	Summary	34
3	Methodology	36
3.1	Environment and Model	36
3.1.1	The Progen Heist Environment	36
3.1.2	Model Architecture	37
3.1.3	Model Training	37
3.2	Interpretability Techniques Employed	38
3.2.1	Sparse Autoencoder (SAE) Training	38
3.2.2	Feature Visualization	39
3.2.3	Steering Vectors and Activation Interventions	39
3.2.4	Ablation Studies	41
3.3	Experimental Design and Evaluation Metrics	41
3.3.1	General Experimental Approach	41
3.3.2	Evaluation Metrics	41
3.4	Summary	42
4	Results	43
4.1	SAE Training and Feature Visualization	43
4.1.1	SAE Performance	43
4.1.2	SAE Feature Visualization Results	43
4.2	Steering and Activation Intervention Results	44
4.2.1	General Steering Capabilities	44
4.2.2	Quantitative Steering Results and Channel Analysis	44
4.2.3	Specific Steering Phenomena	45
4.2.4	Steering with Dead Channels (SAE)	46
4.3	Channel Ablation Study Results (SAE)	46
4.4	Activation-Level Entity Encoding (SAE)	46
4.5	Polysemanticity Observations (Base Model)	47
4.6	Summary of Key Findings	47
5	Discussion	49
5.1	Interpretation of Results	49
5.1.1	The Effectiveness of SAEs for Interpretability	49
5.1.2	The Dead Channel Phenomenon in SAEs	49
5.1.3	Activation-Level Entity Encoding in SAEs	50
5.1.4	Role Specialisation within the SAE	50
5.1.5	Polysemanticity and Goal Representation in the Base Model	51
5.2	Theoretical Implications	51

5.3	Practical Applications	52
5.4	Limitations of the Study	53
5.5	Future Research Directions	54
5.6	Reflections on the Research Process	55
6	Conclusion	56
6.1	Summary of the Research	56
6.2	Key Findings and Contributions	56
6.3	Implications for Theory and Practice	57
6.4	Recommendations	58
6.5	Final Remarks	58

List of Figures

2.1	Sparse auto-encoder (SAE) architecture. An encoder applies a 1×1 convolution and ReLU to map input activations X to sparse hidden activations A ; a second 1×1 convolution reconstructs $\hat{X}$. Training minimises the reconstruction loss $\text{MSE}(X, \hat{X})$ plus an ℓ_1 sparsity penalty $\lambda \ A\ _1$. <i>Shapes:</i> $X \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$, $A \in \mathbb{R}^{B \times C_{\text{hid}} \times H \times W}$, $\hat{X} \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$	26
2.2	The Procgen Heist environment presents a procedurally generated maze where agents must collect keys and unlock corresponding doors in a specific sequence (blue, green, red) before reaching the final goal (gem). The environment varies in complexity, sometimes only having the gem present, and other times including multiple keys and locks.	31
3.1	Visualization of the maze and neural network activations in different layers. Top row (left to right): observation input, layer 1 channel 10, layer 2 channel 10, and activation colorbar. Bottom row: layer 8 channel 29 and layer 9 channel 29. In the top row filters we observe a minimal activation around the regions of the blue key and lock and in the bottom row we see maximal activations around the blue key. This is a replication of the approach used by Mini et al. [2023] in order to get an initial sense of the clarity with which entities could be seen in activations.	40

List of Tables

3.1	Expansion factors and L1 Coefficients for the different SAEs (from Paper 1)	38
-----	--	----

Chapter 1

Introduction

Understanding how deep learning models function is currently an unsolved problem in the field of machine learning. This state of affairs ensures that highly reliable, controllable and understandable models remain out of reach. This is particularly the case in the field of reinforcement learning (RL), which has been somewhat neglected by the techniques of Mechanistic Interpretability (MI) whose focus has largely been on Large Language Models (LLMs) Elhage et al. [2021]. With techniques from RL being applied to frontier models to a greater extent, it would appear that there is a gap in the application of MI techniques to RL models.

Existing techniques are being used to better understand RL models [Atrey et al., 2020], but many techniques developed for MI elsewhere have not yet seen much application in RL (Hilton et al. [2020] being a notable exception). RL does pose natural difficulties in interpretation due to the nature of RL outputs as actions or sequences of actions rather than as words or images. This adds to the challenge of extracting meaningful insights from the earlier layers of the model. The unpredictable nature of RL training further adds to these challenges where the different learning outcomes of the model may lead to confusions about what the genuine nature of what the model has learned can be difficult to determine.

Furthermore, given the high likelihood that RL systems are deployed in the operation of critical infrastructure such as data centres or nuclear fusion reactors Degraeve et al. [2022], there is significant urgency to guarantee high standards of operational safety and show that the policies predictably function within specified operational bounds so that we can avoid catastrophic failures in these systems Amodei et al. [2016a].

There is an opportunity to explore novel techniques for interpreting other categories of AI systems with RL, as RL environments usually contain relatively simple goals and easily determined failure modes that allow for testing whether changes made evoke a desired behavior. For example, the approach of steering Large Language Models (LLMs) using activation steering in Turner et al. [2024a] was inspired by the work done to understand and control a maze solving agent in the Procgen Maze environment Mini et al. [2023].

The investigation seeks to build on the work of Mini et al. [2023], who found key parts of the model that could be used to control model behavior. This investigation’s contribution is to extend their work to an environment that involves multiple competing entities that an agent needs to reach in sequence. To this end, the Procgen Heist environment was chosen as the target of this research. This is because the procgen environment requires the agent to navigate through three locks of different colours each requiring a corresponding key before reaching a gem at the end. This means that the agent has to learn robust mechanisms for navigating to each entity in the environment [Cobbe et al., 2020]. By observing how the model learns to approach multiobjective maze solving, this work aims to understand how neural networks represent and pursue multiple sequential goals.

A significant challenge in understanding the internal mechanisms of complex neural networks, particularly those operating in multifaceted environments like Procgen Heist, is the prevalence of polysemanticity Elhage et al. [2022]. This phenomenon, where individual neurons respond to multiple distinct features, can make it challenging to draw strong conclusions about how a model works. To address this, Sparse AutoEncoders (SAEs) can be employed. SAEs are a type of unsupervised learning technique that can be used to break down the most critical functions of a model into a more interpretable set of sparse latents, often by training an overcomplete basis on the original activations of a given layer of the network Bricken et al. [2023]. In this study, we apply this approach to the convolutional neural network (CNN) layers to disentangle model functionality. Our methodology for adapting SAEs to CNNs builds upon recent work by Gorton [2024], and will be described in later sections.

1.1 Background and Motivation

RL is a field of machine learning that has the unique promise of delivering superhuman performance in a wide range of environments Silver et al. [2016]. However, the black box nature of these models has made it difficult

to understand how they achieve these results. In recent years, RL has seen a surge in interest from the ML community, due to its application in LLMs and other frontier models Ouyang et al. [2022], Bai et al. [2022]. This has led to significant advances in performance in these models, but also presents the potential for these highly capable systems to manifest unexpected behavior that is unique to the RL setting [Krakovna et al., 2020].

Machine learning as a field is composed primarily of three different approaches to training models. These are supervised learning, unsupervised learning, and RL. Supervised learning used explicitly labeled data to allow models to learn the underlying patterns that connect the input to the specific label. Unsupervised learning uses no labels and simply has the model learn structures by identifying underlying patterns in the data.

RL differs in that it focuses on the idea of an environment that is the source of truth for the agent. By providing actions to the environment based on its policy, the agent receives new information in the form of observations from the environment, as well as reward information that provides a signal about whether it is achieving the goals that the designers of the model and environment intended Sutton and Barto [2018]. This is typically done by using the reward as a key variable in the loss function, with higher rewards being linked to lower loss. Though for some approaches such as temporal difference learning, the loss is reduced by minimising the difference between the predicted and actual value of the next state Sutton and Barto [2018].

This means that with relatively low amounts of human effort in shaping environment behaviour, agents trained with RL can achieve performance that can far surpass human capabilities in a domain, as was demonstrated by systems such as [Silver et al., 2016] AlphaGo. In that case the agent was trained through self play whereby it played against a similarly skilled version of itself, and would learn to improve on its mistakes based on whether the moves it made in a game led to victory or defeat.

Modern RL often employs deep neural networks in order to achieve some of its most remarkable results, but this leads to the issue of the lack of interpretability into how those results are achieved due to the black box nature of modern neural networks.

The field of MI seeks to help decode the black boxes of neural networks so that these models can be used with greater confidence and reliability, and to allow for proactive identification of issues in models' behaviour before critical issues arise Olah et al. [2018].

MI as a field aims to answer the question of how neural networks actually

function at a very granular level. In the ideal case it would make it possible to reverse engineer a given neural network into a human readable algorithm that can be used to generate the same outputs as the network, through a complete knowledge of how its parts work. Having this understanding would provide a broad set of tools for helping to make neural networks safer, and more reliable.

MI broadly refers to a number of approaches that can be used to gain insight into neural network operations, from visualising neurons and learned filters in CNNs, to the training of specialised neural networks that help better understand the operations of a particular part of a network.

The ultimate goal of MI has become to find ways to ensure the safety of AI systems, and this has become particularly critical with frontier agentic AI that has the ability to perform long horizon tasks that could involve taking actions that are highly impactful to humans. It is critical to be able to understand and prevent unwanted intent from developing in such systems, and to do this it is necessary to understand what goals the system may have.

For a long time, many in the field questioned whether models had an internal model such that it would even make it coherent to discuss whether a neural network could have things such as ‘intent’ or ‘goals’. However recent work has provided compelling evidence that neural networks can indeed learn meaningful internal models and conceptual abstractions from data. For instance, studies on language models trained only on the moves made in a game such as Othello, with no explanation of the rules, revealed that these models spontaneously developed internal representations of the game state without explicit supervision of the rules[Li et al., 2022].

Other research has shown that such models naturally learn geographic mappings of the world related to locations that can be retrieved directly from the activations of the network, effectively building simplified world models’ [Gurnee and Tegmark, 2023]. Similarly, analyses of agents trained on complex games like chess have demonstrated that these systems acquire human-understandable strategic concepts, such as piece values or tactical motifs, purely from self-play [McGrath et al., 2022]. These findings suggest that the internal workings of these models are richer than previously thought and lend support to the idea that they may plausibly learn to represent goals that can be directly extracted from the model and understood.

It therefore seems plausible that something as fundamental as goals would be likely to share some properties in the way it is encoded across models. With that hope in mind, this work seeks to reveal some of these properties

to inform investigations across future models.

However not all models which seek to target goals are good candidates for this research, as it may easily be the case that the task is simple enough to solve without the need to develop a sufficiently complex mechanism to represent abstract concepts like goals. The procgen environments stand out as good environments for this reason, as they procedurally generate environments for the agent to solve. This means that the agent cannot simply overfit to the data but has to learn general patterns in order to score well on the environment, which would necessitate developing a complex representation of goals within the model.

The choice to use Procgen Heist in particular stemmed from the desire to explore how multiple goals might be represented in the model, as significant work has already shown the presence of ‘cheese representations’ with the Procgen Maze environment[Mini et al., 2023]. Because advanced agentic AI would be operating environments with potentially multiple goals, the question of whether there was a substantial difference in the representation of these goals was an essential one.

However, in making the environment more complex the model too became more complex, and the problem of polysemantic neurons became a significant hurdle to understanding the model’s operations. Specifically whether the model’s representations of goals in the base model represented a ‘true’ representation or whether it was simply the results of polysemanticity in the network.

Polysemanticity refers to the case of a single neuron having multiple coherent feature representations encoded simultaneously, due to it needing to encode many such features but not having sufficient neurons to represent them all. In such cases the model will compress multiple features into a single neuron, typically encoding features that are unlikely to activate strongly simultaneously in practice to avoid causing harmful interference.

This led to the use of Sparse AutoEncoders (SAEs) to confirm whether neurons in the network would naturally tend to represent the different entities using different neurons, or whether there were single ‘current goal’ channels in the network that would naturally come to exist.

An SAE is a model trained with an unsupervised approach to optimally reconstruct the outputs of a given layer in the network. Typically it is trained with a loss function that minimises the reconstruction loss, and the L1 norm of the network, which means that it produces the smallest possible activations. Typically the SAE has a hidden layer between an encoder and decoder

which are the primary parameters of the network, and the size of the hidden layer is normally based on the size of the original layer multiplied by some expansion factor. This means that the SAE learns an overcomplete basis of the original layer, which allows for the separation of polysemantic features into separate neurons in the network, which can then be analysed in isolation Bricken et al. [2023].

Being able to understand how these models work is a key step towards building more reliable, controllable and understandable models. Specifically, there is interest in understanding how these models represent and pursue goals in complex, multi-objective environments that are a step beyond simple toy environments. Goals, specifically are something that advanced agentic AI would share with even simple models, and so greater knowledge on the underlying principles behind this universal concept may provide fundamental insights that generalise beyond toy environments.

1.2 Problem Statement

As AI systems become more complex and are deployed more widely, it will become increasingly necessary to provide safety guarantees and explanations behind critical trade-offs that RL policies are making so that these systems can be deployed with confidence that they will work as expected [Shakya et al., 2023].

If we are unable to understand how goals specifically are represented and pursued within RL models, we risk deploying systems that may pursue unintended or misaligned objectives, leading to unpredictable or harmful outcomes, especially in complex, multi-step tasks. This challenge is exacerbated by the inherent opacity of deep neural networks, where phenomena such as polysemanticity, a key focus of this investigation, can obscure the goal-related features we wish to isolate and understand. Without a clear understanding of these internal goal representations, ensuring agent alignment with intended behaviors, debugging undesirable policies, and building reliable autonomous systems remains a formidable task. The lack of such understanding in the context of multi-objective sequential decision-making, as typified by environments like Progen Heist, forms the central problem this thesis aims to address. Addressing this problem necessitates a direct inquiry into the nature of these representations and the methods to uncover them, leading to the following research objectives.

1.3 Research Objectives

This research seeks to demonstrate that it is possible to determine the key functional components in a maze solving agent which are responsible for determining the agent’s goal. To this end, our specific objectives are to:

- Determine what parts of the model are uniquely responsible for abstract goal formulation and how goals are represented in the model weights. This includes investigating whether the model captures goals with singular representations within channels, or if there are general **current goal** or entity-specific steering channels in the network.
- Assess the utility of Sparse AutoEncoders (SAEs) in disentangling polysemantic activations to better determine the role of specific, interpretable features in goal representation.
- Investigate the mechanisms by which certain SAE channels, including those with low baseline activity (so-called **dead** channels), can nonetheless influence or steer the network’s behavior when subject to interventions.
- Explore the factors contributing to observed differences in success rates when attempting to steer the agent’s behavior away from various entities or objectives within its environment.
- Evaluate how effectively SAEs preserve the agent’s task performance while modifying the original model’s activations, thereby aiming to ensure that the features learned by the SAEs are relevant to the agent’s operational policy.
- Identify whether different neural network layers, either in the base model or in the derived SAEs, exhibit specialized roles in the processing of goal-related information or in their susceptibility to steering interventions.

Through these objectives, particularly by leveraging SAEs, we aim to provide clearer evidence of how the model learns, represents, and pursues sequential goals in an environment such as Procgen Heist.

1.4 Research Questions

This research seeks to answer the following key questions:

1. How are goals represented in the model weights?

2. Can we use Sparse AutoEncoders (SAEs) to better determine the role of specific features?
3. Why do dead channels seem able to still steer the network?
4. Do specific channels target specific entities, or are there general goal steering channels in the network?
5. Why do we observe different rates of success when steering the agent away from different entities?
6. Given that SAEs modify the original model's activations, how effectively do they preserve the agent's task performance, ensuring that their learned features are relevant to the agent's behavior?
7. Does the research indicate that different neural network layers (in the base model or SAEs) play specialized roles in how the agent processes goal information or becomes steerable?

1.5 Key Hypotheses

Based on the preceding research questions, this study will investigate the following key hypotheses:

1. It is hypothesized that the model predominantly learns to represent specific goals (like individual keys or the gem) via dedicated channels, rather than relying solely on generalized 'current target' signals or highly distributed, non-specific encodings.
2. It is hypothesized that Sparse Autoencoders (SAEs) can deconstruct complex, polysemantic activations within the agent's layers into a more disentangled and interpretable set of features, thereby clarifying the specific roles of underlying neural computations related to goals.
3. It is hypothesized that the observed influence of apparently 'dead' or low-activation SAE channels during steering interventions arises from the residual impact of their learned decoder weights.
4. It is hypothesized that variations in the success rates of steering the agent away from different entities are linked to the amount of time that the model spent in direct observation of those entities.
5. It is hypothesized that SAEs can be trained to reconstruct the original model's activations with sufficient fidelity to preserve the agent's task

performance, indicating that the features learned by the SAE are indeed relevant to the agent’s operational policy.

6. It is hypothesized that distinct layers within the Procgen agent’s network (and by extension, the SAEs trained on them) play specialized roles in processing goal-related information, potentially with earlier layers focusing on perceptual parsing of the location of entities and the structure of the maze, while later layers encode more abstract things such as which entity to pursue plans for how to reach them.

1.6 Scope and Limitations

This work is limited by the scope of the Procgen Heist environment, which is a simplified environment that may not fully capture the complexity of real-world decision-making in sequential, multi-objective tasks. It is also limited by the use of a single model architecture, and the use of a single environment.

Furthermore, the reliance on Sparse Autoencoders (SAEs) as the primary interpretability tool introduces its own set of limitations. The features discovered by SAEs can be sensitive to hyperparameter choices, such as the sparsity coefficient and dictionary size, which can influence the nature of the learned representations. This can create a gap between the features the SAE learns and the true underlying computational nature of the network. Additionally, the nature of the RL environment itself poses a challenge; much of the model’s complexity is likely encoded in functionally critical but semantically “illegible” factors, such as subtle patterns for navigating obstacles. While SAEs may successfully isolate high-level goal features, they may struggle to render these low-level navigational strategies intelligible. Consequently, interpreting the full suite of discovered features remains a significant challenge, and SAEs may not capture all of the original model’s computational strategies.

While the environment is a good fit for this research, its complexity means that the construction of experiments is quite time consuming, which is the reason why only this environment was used.

There are also a number of cutting edge MI techniques that became available over the course of this research, such as the use of bilinear activation functions to do weight based interpretability, and parameter based decomposition, which allows for the decomposition of a given channel into a sum of other activations, which can then be individually analysed and understood. Due to the time constraints of the research, these were not explored.

1.7 Significance of the Study

This research contributes to the field of MI by demonstrating the application of Sparse Autoencoders, a technique largely developed for language models, to the distinct challenges of Reinforcement Learning agents. Understanding goal representation is fundamental to ensuring agent alignment, especially as RL is increasingly used in safety-critical and frontier AI systems. By providing a pathway for disentangling and interpreting how an RL agent represents its objectives, this work offers new techniques and insights for decoding the internal mechanisms of these models.

Furthermore, the study’s findings have broader implications into how different entities within an environment are captured by CNN models. The investigation into phenomena like ‘dead channel’ steering contributes to our fundamental knowledge of both neural network function and the properties of our interpretability tools. Some of the experimental frameworks used provide a replicable template for future studies seeking to causally probe goal-directed behaviors in neural networks. Ultimately, such insights yield new research directions on how environmental factors shape learning dynamics and help inform the development of more robust and reliable AI systems.

1.8 Thesis Structure

This thesis is structured as follows: Chapter2 provides a review of the relevant literature in RL and MI. Chapter3 details the experimental setup, including the environment, model architecture, and the interpretability techniques employed. Chapter4 presents the findings from the experiments, including feature visualizations, steering results, and ablation studies. Chapter5 discusses the implications of these findings, and Chapter6 concludes the thesis and suggests avenues for future research.

Chapter 2

Literature Review

2.1 Introduction

The previous chapter established the motivation for this research, highlighting the need to better understand the internal workings of Reinforcement Learning (RL) agents, particularly in multi-objective environments. This chapter provides a review of the foundational literature that underpins this thesis.

This review will cover the fundamentals of RL relevant concepts in mechanistic interpretability (MI). It will then delve into the specific challenges and recent advancements in making RL agents more understandable. In particular we examine the foundational works upon which this research rests from the fundamental RL concepts to the algorithms used to train the models. The rest of this section will situate the present research within the research landscape and seek to demonstrate how it builds upon prior work in the field of MI to address unsolved problems in RL interpretability.

Chapter 3 builds on this review, translating the concepts reviewed here into approaches that were employed in the remainder of the thesis.

2.2 Reinforcement Learning Fundamentals

This section provides a brief overview of the fundamental concepts in RL that are relevant to the research discussed. While a baseline understanding of RL and AI concepts is assumed, key elements of the environments and training paradigms employed are highlighted.

2.2.1 Markov Decision Processes

At its core, the problem that RL solves in general can be formalized as a Markov Decision Process (MDP). An MDP is a mathematical framework for modeling decision-making in situations where outcomes are partially random and partially determined by the policy of the agent. It is defined by a tuple (S, A, P, R, γ) , representing the set of states, the set of actions, the state transition probability function $P(s'|s, a)$, the reward function $R(s, a, s')$, and a discount factor $\gamma \in [0, 1]$ that balances immediate and future rewards. The agent’s goal is to learn a strategy that maximizes the cumulative discounted reward over time. Because the agent receives the entire 64 by 64 pixel view of the maze as its observation, the Procgen Heist environment is a fully-observable MDP, where the state is directly represented by the agent’s visual input. There are variations of the environment which involved a limited view of the environment such as the ‘memory’ distribution, which would be considered a partially observable MDP (POMDP).

2.2.2 Value Functions and Policies

The process of training a model involves iterating through many interactions with the environment to produce a policy denoted as $\pi(a|s)$, which is a mapping from states to a probability distribution over actions. The goal of the RL training process is to find an optimal policy, π^* , that maximises the agent’s expected reward over time. As a means of estimating the quality of a given policy, RL functions often use value functions Sutton and Barto [2018]. The state-value function, $V^\pi(s)$, represents the expected return after taking an action a in state s based on following policy π . The action-value function, $Q^\pi(s, a)$, is the expected return from continuing to follow policy π after taking an action given a certain state.

These functions are foundational to the function of training algorithms such as Proximal Policy Optimisation (PPO, discussed in Section ??), which alternates between sampling data from the environment and optimising a ‘surrogate’ objective via stochastic gradient ascent.

2.2.3 Deep Reinforcement Learning

Deep Reinforcement Learning (DRL) combines deep neural networks with RL principles to learn effective policies in complex, high-dimensional environments like Procgen. These models typically use convolutional neural networks (CNNs) to process visual input and feed the resulting features into fully-connected layers that output a policy.

The research this work is based on utilizes specific DRL architectures chosen for their balance of performance and interpretability. Following the precedent set by Hilton et al. [2020], which demonstrated that simpler architectures often yield more interpretable representations without significant performance loss, the models used were simplified versions of established architectures. For example, the primary policy model used is a compressed version of the Impala architecture [Espeholt et al., 2018] with only 5 convolutional layers instead of the standard 15. This choice reflects the principle that simpler architectures are often easier to interpret [Hilton et al., 2020].

IMPALA (Importance Weighted Actor Learner Architecture) was introduced by as a scalable RL framework, but its convolutional encoder has since become a primary baseline for Procgen and Atari agents [Espeholt et al. [2018]]. The encoder consists of three “IMPALA blocks”, each containing:

- A 3×3 convolution with stride 1 followed by ReLU,
- A second 3×3 convolution and ReLU,
- 2×2 Max-pooling (stride 2), which halves the spatial resolution.

Every block doubles the channel count (from $16 \rightarrow 32 \rightarrow 64 \rightarrow 128$ in the original), though in work such as that performed by Mini et al. [2023] the number of channels remains fixed. A final fully-connected layer produces a 256 dimension embedding that feeds into two heads, one which outputs the actions of the policy, and the other producing estimated value.

Earlier work also shows that training dynamics of RL systems can have a significant impact on the interpretability and clarity of weights in the trained network. Engstrom et al. [2020] show that doubling the rollout length (e.g. from 128 to 256 steps) stabilises gradients, which in turn makes gradient-based visualisations such as saliency maps crisper, as later confirmed by Thatte et al. [2022]. This has important consequences for potential MI work.

2.3 Reinforcement Learning Safety

2.3.1 Safety Problems in RL

RL suffers from several well-documented failure modes, notably reward hacking, unsafe exploration, distributional shift, and goal misgeneralisation [Amodei et al., 2016b, García and Fernández, 2015, Langosco et al., 2022].

These pathologies often surface in surprising ways. During training, ex-

ploratory actions are frequently close to random; if such actions were executed on a physical robot they could easily damage themselves or their surroundings. Moreover, entirely novel stimuli are typically treated as reward-neutral by default, making it non-trivial to teach agents caution around unknown phenomena[García and Fernández, 2015].

Reward hacking occurs when an agent finds a loophole that maximises the reward signal without producing the behaviour we truly want. A well-known example is the CoastRunners Atari agent that learned to drive in circles collecting speed boosts rather than racing, because this strategy yielded higher reward[Krakovna]. Such outcomes are typical instances of specification gaming, where a proxy objective is optimised at the expense of the real goal[Amodei et al., 2016b].

In the real world the problem of safe exploration is equally pressing: naïve random exploration can damage the agent or its environment. Research on risk-sensitive and shielded RL provides principled ways to constrain exploration steps[García and Fernández, 2015, Kahn et al., 2017].

Even if exploration is safe and training rewards are faithful, agents can fail under distribution shift. Policies that perform well in simulation or under limited training conditions often generalise poorly when the environment changes[Amodei et al., 2016b]. Progen itself has been introduced as a benchmark to quantify such generalisation gaps [Cobbe et al., 2019], which makes it a natural testbed for the present thesis.

Relatedly, goal misgeneralisation describes policies that appear successful during training but pursue the wrong goal when deployed outside of their training distribution[Langosco et al., 2022].

This gap between performance on the training distribution and robust, safe behaviour has motivated a significant body of work arguing that mechanistic interpretability is a critical component of ensuring RL safety [Hubinger et al., 2019, Olah et al., 2018]. The premise of this research is that by directly inspecting the learned representations and algorithms inside a policy network, it may be possible to move from post-hoc observation of failures to a predictive understanding of their causes. For instance, identifying that an agent’s internal concept of a ‘goal’ is conflated with distractor features could help diagnose goal misgeneralisation before deployment. In this view, MI tools are seen not just as instruments for scientific curiosity, but as necessary for verifying the alignment and robustness of RL systems intended for real-world application.

2.4 Interpretability in Machine Learning

Interpretability in machine learning refers to the extent to which a human can understand the cause and effect of a model’s decisions. It is crucial for debugging, ensuring fairness, building trust, and verifying reliability, especially in high-stakes applications.

One significant challenge that arises when doing MI is that representations of concepts within neurons are often not clean singular representations, but instead those representations may be smeared across multiple neurons or have multiple representations encoded within a single neuron. The latter case is typically called polysemanticity. This refers to the tendency of individual neurons or channels to fire for multiple, seemingly unrelated features. This was first publicly observed in the ‘curve-circuits’ analyses of Inception V1, where single units responded to several distinct curve orientations [Cammarata et al., 2020].

This phenomenon is now understood as a consequence of superposition. Because deep layers have limited dimensionality, networks compress many latent directions into overlapping linear combinations, so a neuron’s activation mixes several underlying features [Olah et al., 2020, Elhage et al., 2022]. This typically happens with features that are unlikely to fire under the same input distribution, such as car doors and hummingbird wings [Olah et al. [2017]]. While efficient for representation learning, polysemanticity poses significant challenges for mechanistic interpretability, making neuron-level ablations or visualisations hard to attribute to a single concept.

Let $x \in \mathbb{R}^d$ be the activation vector of a layer expressed as a linear combination of k (potentially $k \gg d$) latent feature directions collected in $F \in \mathbb{R}^{d \times k}$, i.e.

$$x = F s, \quad s \in \mathbb{R}^k, \text{ sparse.} \quad (2.1)$$

Because $k > d$, the same neuron must participate in multiple columns of F , giving rise to *polysemantic* ‘units’. Defining a linear read-out $a = W^\top x$ with $W \in \mathbb{R}^{d \times d}$, we obtain

$$a = W^\top F s =: M s. \quad (2.2)$$

A neuron i is polysemantic when the i -th row of the mixing matrix M has significant weight on multiple features (row sparsity $\|M_{i,:}\|_0 > 1$). The toy model of Elhage et al. [2022] shows that packing many sparse features (k large, s sparse) into a narrow layer of width d forces an unavoidable trade-off:

$$\mathbb{E} \|a - M s\|^2 \propto \frac{\lambda k}{d},$$

where λ is the expected sparsity of s . Methods such as Sparse AutoEncoders (SAEs) reduce the effective λ by encouraging even sparser codes, thereby converting polysemantic ‘units’ into more interpretable, monosemantic directions.

Recent approaches—including Sparse Autoencoders (SAEs) [Bricken et al., 2023, Gorton, 2024], sparse probing [Dalvi et al., 2021], and neuron-level interventions [Meng et al., 2023]—combat superposition by learning over-complete sparse dictionaries or low-dimensional subspaces that disentangle features. In particular, SAEs compress activations into sparse, over-complete codes that split polysemantic neurons into monosemantic latents and have revealed interpretable features in both CNNs and transformers [Conerly et al., 2024].

SAEs provide a sparse encoding of the original features of the model (which we refer to as latents in the SAE), which arise due to the enforcement of a sparsity penalty applied during the the training of the SAE. SAEs aim to faithfully reconstruct the activations of a given layer in a network while keeping their own internal activations sparse. The training objective is therefore to find encoder and decoder weights that minimize a loss function combining a reconstruction term and a sparsity term, which typically takes the form:

$$L(x) = \|x - \text{Dec}(\text{Enc}(x))\|_2^2 + \lambda \|\text{Enc}(x)\|_1$$

Here, x represents the original activation from the base model, the L2 norm penalizes poor reconstruction, and sparsity is enforced by minimising the product of the L1 norm of the SAE’s internal activations, and a coefficient λ [Bricken et al. [2023].

SAEs typically learn an overcomplete basis, where the hidden layer of the SAE is much larger than the original layer it is reproducing, typically chosen by multiplying the original hidden dimension size by some expansion factor. This high dimensionality allows the SAE to represent the input activations using a small number of highly specific, disentangled features. This addresses the problem of polysemantic ‘units’ in a model, which can make the task of accurate interpretability much more challenging [Elhage et al. [2022].

The direct methodological precedent for applying SAEs to convolutional networks comes from recent work by Gorton [2024], who successfully trained SAEs on the InceptionV1 image classifier. Their approach uses a ‘1x1-conv SAE’ architecture, as illustrated in Fig 2.1. An encoder with a learnable 1×1 kernel maps the input activation tensor X to a sparse, high-dimensional hidden activation tensor A , which a symmetric 1×1 decoder then uses to

reconstruct $\hat{X}$. To train this effectively on CNNs, they introduced several key innovations. First, to manage computational cost, they used *activation over-sampling*, preferentially sampling training data from spatial locations with high activation magnitudes to focus on learning meaningful features rather than background noise. Second, they used highly over-complete dictionaries, with expansion factors up to 8, to provide the capacity needed to disentangle features. The results were compelling: their SAEs successfully split known polysemantic ‘double-curve’ units in InceptionV1 into distinct monosemantic features and recovered a suite of previously ‘missing’ curve detectors. These findings provide concrete evidence that dictionary-learning SAEs can resolve superposition in CNNs and motivate their application to RL agents with convolutional backbones.

2.4.1 Visualising neural networks

Techniques such as those developed by Zeiler and Fergus represented some of the first work on interpreting the operations of deep neural networks. They employ a deconv-net, a model that maps feature activations from a deep layer back to the input pixel space by approximately reversing the network’s pooling and convolutional operations. By reconstructing the maximally activating zones in pooling layers and being chained through subsequent CNN layers the reconstruction of complex visual representations of deep layers within the network can be achieved. This is normally very challenging due to the significant compression that is performed on the deep layers in the CNN. By applying this deconvolution the features appearing deeper in the network can be expanded back up to human interpretable scales.

Early work in understanding the functioning of neural networks focused on techniques that could visually represent the features that were encoded in the weights of the neural network. In particular Olah et al. [2017] developed techniques to generate maximally activating synthetic inputs that functioned as startlingly clear representations of channels and layers within the network. These visualisations revealed a clear hierarchy of abstraction, demonstrating how simple features from one layer (‘curve-circuits’) compose into more complex, semantically coherent representations (e.g., object parts) in the next. This work also demonstrated the use of gathering maximum activating samples from the training data and pairing it with particular channels and showing the similarity between these and the synthetically generated examples as a means of confirming the veracity of the visualisations.

While synthetic visualizations can be noisy, maximally activating patches, especially from within the environment, can yield interpretable results, con-

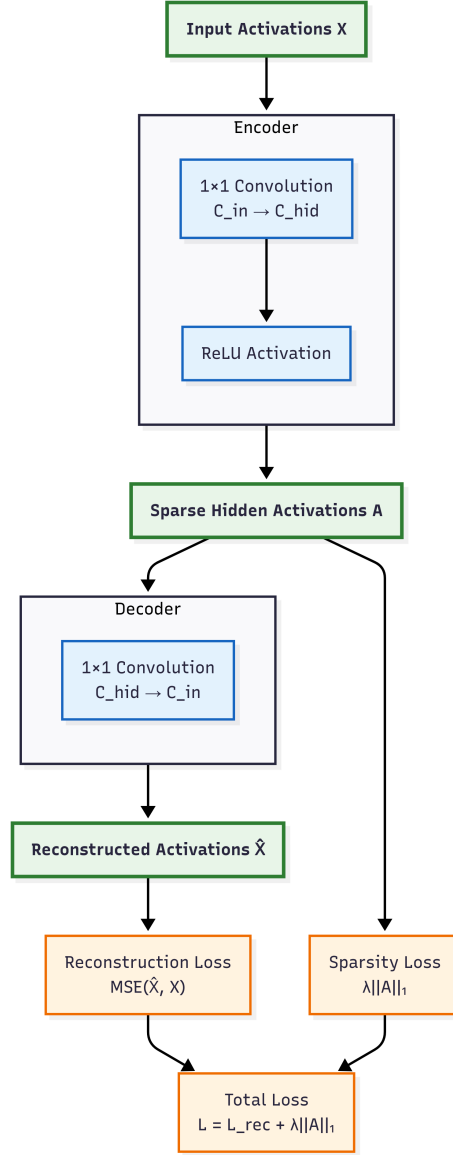


Figure 2.1: Sparse auto-encoder (SAE) architecture. An encoder applies a 1×1 convolution and ReLU to map input activations X to sparse hidden activations A ; a second 1×1 convolution reconstructs $\hat{X}$. Training minimises the reconstruction loss $\text{MSE}(X, \hat{X})$ plus an ℓ_1 sparsity penalty $\lambda \|A\|_1$. *Shapes:* $X \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$, $A \in \mathbb{R}^{B \times C_{\text{hid}} \times H \times W}$, $\hat{X} \in \mathbb{R}^{B \times C_{\text{in}} \times H \times W}$.

firming findings in Hilton et al. [2020]. Work done on exploring SAEs trained on CNNS such as Gorton [2024] makes effective use of visualisation to illustrate the improvement in interpretability of the SAE latents.

2.4.2 Post-hoc Explanations

Attribution and saliency methods. Post-hoc explanation methods aim to provide insights into already trained models without altering their architecture or training process. Examples include feature attribution methods (like LIME, SHAP), saliency maps, and activation steering.

Probing. Perhaps the most famous and enduring of this type of technique is probing, particularly the linear probe. This is simply a linear classifier trained on activation data from a given layer on target labels that are provided. The degree of accuracy that the probe can achieve represents the degree to which a given layer can be said to have a clear representation of the given target feature within the layer Alain and Bengio [2018]. Multi layer probes are also a useful tool, but these can obfuscate the degree to which the target feature is actually present due to the additional computation being performed.

Concept Activation Vectors (CAVs). Concept Activation Vectors provide a way to bridge high dimensional vector space of neural network activations and human-readable concepts. To construct a CAV for a concept (‘striped’), one collects a small set of images that exhibit the concept and a set of random counter-examples, records the activations from a chosen layer, and trains a linear classifier to separate the two groups. The normal vector to the resulting decision boundary is the CAV. This can then be manipulated in one direction or the other to produce more or less of the effect you desire Kim et al. [2018]. This can be used to test how much a particular concept is relied upon when the network is making a prediction.

To formalise the above concept, let $a^l(x) \in \mathbb{R}^d$ be the activations of layer l for input x , and let $w \in \mathbb{R}^d$ be the extracted weight vector of the linear classifier trained to distinguish positive concept examples from counter-examples. We take the concept activation vector as

$$v_C^l = \frac{w}{\|w\|_2}.$$

Following Kim et al. [2018], the extent to which concept C influences the

logit $f_k(x)$ for class k is measured by the directional derivative

$$S_{C,k,l}(x) = \nabla_{a^l(x)} f_k(x) \cdot v_C^l,$$

whose sign indicates whether amplifying the concept increases ($S_{C,k,l} > 0$) or decreases ($S_{C,k,l} < 0$) the likelihood of a particular prediction.

Because these can be learned with a lightweight linear probe and then used to potentially intervene on neural network activations as illustrated in work such as othello GPTNanda et al. [2023], this technique has become fundamental as a tool in alignmentMeng et al. [2023], and lays a conceptual foundation for things like steering vectors, which are produced in a different way but share many propertiesTurner et al. [2024b].

Activation steering, as demonstrated inMini et al. [2023] and extended in the discussed research, is a post-hoc technique. Steering vectors, representing the difference in activations between states with and without an objective, can be used to influence a policy model’s behaviour by acting as a superstimulus of sorts of a particular aspect of the environment to the model. This can be achieved through the simple addition of the steering vector into the given layer from which it was produced after multiplying it by a given factor to modulate its effects.

The feature visualization methodologies discussed, such as synthetic visualization (optimizing an input image for maximal channel activation) and finding maximally activating dataset samples [Olah et al., 2017, Hilton et al., 2020], are also forms of post-hoc analysis.

2.4.3 Intrinsically Interpretable Models

Intrinsically interpretable models are designed from the outset to be understandable, often by constraining their complexity (e.g., linear models, decision trees) or by building in mechanisms that are inherently transparent. Training SAEs can be seen as an effort to transform parts of a complex model into a more intrinsically interpretable representation by disentangling features.

Many existing interpretability techniques were developed for supervised settings with i.i.d. data. Reinforcement learning violates many of these assumptions. For example, observations are generated by an agent acting in, and affecting, a dynamic environment, and credit must be assigned across time. As a result, many tools require adaptation to work within this paradigm. The next section explores some techniques that have been developed explicitly to overcome this gap.

2.5 Interpretability in Reinforcement Learning

Interpretability in RL presents unique challenges due to the agent’s interaction with an environment, the fact that specific outputs of the model might be extremely semantically distant from the contents of a given activation, and because we can only infer the meaning of a given observation of an activation within the model, we can’t directly glean it as we could a word extracted from the embedding space of a language model.

As illustrated once more in Fig 2.2, even a seemingly simple maze can embed multiple, dynamically changing goals whose representations the agent must learn to use effectively.

Work byBetley [2023] found a specific circuit in the convolutional layers of the Impala model that activated strongly when the direction to go was ‘up’ and didn’t activate when the goal was in a similar spot but the agent would need to go in a different direction. This indicates that much of the ‘planning’ was already handled in the CNN layers, suggesting it should be possible to train probes that can detect the exact direction the agent needs to move towards.

The larger circuits framework byOlah et al. [2020] provides methods to understand how neural networks combine simple features into complex representations. Their work on polysemantic ‘units’ (neurons responding to multiple distinct features) is particularly relevant when an agent must represent multiple objectivesElhage et al. [2022].

Saliency-based feature-attribution techniques, while originally developed for image classification, have been shown to yield insights into RL policies. These methods typically operate by computing the gradient of a relevant policy output (such as the value of a state or the probability of a specific action, $f(s)$) with respect to the input observation, s . The resulting saliency map is then defined as the magnitude of this gradient:

$$M(s) = |\nabla_s f(s)|,$$

which highlights pixels that most strongly influence the agent’s decision. Using this approach,Greydanus et al. [2018] applied gradient saliency maps to Atari DQN agents, showing that agents frequently track critical entities like the ball in Breakout or enemy ships in Space Invaders. To address the shortcomings of simple gradients, Sundararajan’s integrated-gradients methodSundararajan et al. [2017], which accumulates gradients along a path

from a baseline input, was later extended to policy networks by Atrey et al. [2020], who introduced ‘policy saliency’ maps that operate over distributions of outputs from policy models. These approaches provide an easily computable and highly informative result without requiring additional training. However, unlike SAEs or steering vectors, these methods do not provide a mechanism for manipulating behaviour.

Work exploring the Procgen Coinrun environment builds on saliency based methods to calculate an attribution within the model weights themselves. By measuring the contribution of a given convolutional feature map to the value function, it becomes possible to determine if a particular feature that the model is observing is likely to have a positive or negative effect on the success of the model, based on whether the collected gradient multiplied by the activation score is positive or negative. This advances previous saliency mapping methods by linking the gradients to specific features within the model, rather than pixels in the input Hilton et al. [2020].

This is achieved by applying non-negative matrix factorisation (NMF) of the attribution tensor to cluster correlated locations across channels into object-level ‘features’ such as coins, enemies, obstacles etc. By using this dimensionality reduction they are able to produce clear, interpretable object detectors.

To provide a precise explanation of this process, let $X^{(n)} \in \mathbb{R}^{C \times H \times W}$ be the activation tensor of a convolutional layer for input n and let $V^{(n)}$ be the value head. Element-wise product of activation and gradient attribution is

$$A_{c,h,w}^{(n)} = X_{c,h,w}^{(n)} \frac{\partial V^{(n)}}{\partial X_{c,h,w}^{(n)}}.$$

Flatten $A^{(n)}$ to a vector $a^{(n)} \in \mathbb{R}^D$ with $D = CHW$ and stack N such vectors into a matrix $R \in \mathbb{R}^{N \times D}$. This results in an $N \times D$ matrix where each row is a single observation, and each column is a ‘feature location’. We then apply NMF to this matrix, keeping $A_{c,h,w}^{(n)+} = \max(0, A_{c,h,w}^{(n)})$ to satisfy the non-negativity constraint. Non-negative matrix factorisation seeks

$$R \approx WH, \quad W \in \mathbb{R}_{\geq 0}^{N \times r}, \quad H \in \mathbb{R}_{\geq 0}^{r \times D},$$

with a small rank r . Each row of H , reshaped back to (C, H, W) , yields a sparse mask that acts as a detector for in-game entities (coin, enemy, hazard).

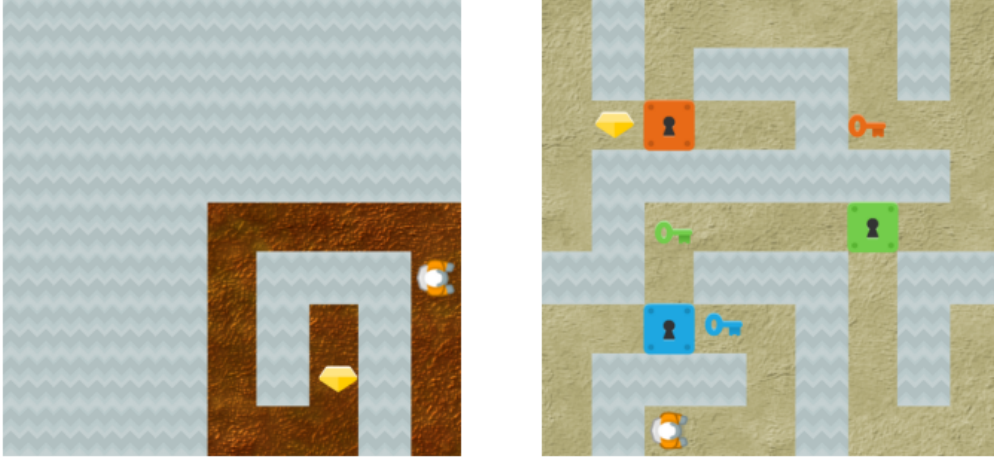


Figure 2.2: The Progen Heist environment presents a procedurally generated maze where agents must collect keys and unlock corresponding doors in a specific sequence (blue, green, red) before reaching the final goal (gem). The environment varies in complexity, sometimes only having the gem present, and other times including multiple keys and locks.

2.6 The Progen Suite of environments

The Progen suite of environments consist of 16 different environments that rely on procedural generation to create diverse challenges for RL agents to learn to solve. They are inspired by Atari games and thus build upon existing games such as those used in the Arcade Learning Environment Bellemare et al. [2013]. This type of environment forces the agent to not overfit to a particular solution for playing the games effectively, and instead forces it to learn a robust, generalisable policy for each environment.

The Progen Heist environment, the focus of the presented research, requires an agent to collect up to 3 keys and 3 locks before reaching the final goal, a gem Cobbe et al. [2020]. The order in which the keys need to be collected is always the same (blue, green, red). The environment is procedurally generated and might generate with any combination of no keys, or 1–3 keys, as can be seen in 2.2. The size of the maze will also change depending on how many objects are included. The environment provides sparse rewards, only giving out a score of 10 if the agent makes it all the way to the final goal and 0 otherwise. This establishes a natural curriculum for the agent and gives it a reasonable chance of reaching the end with a random walk before it starts to develop an actual policy.

This environment was selected due to the necessity for the agent to pursue multiple objectives within a single episode, likely influencing the development of generalizable maze-navigation circuits. It is a sparse learning environment.

2.7 Understanding and Controlling a Maze Solving Policy Network

Previous work by Mini et al. [2023] demonstrated the ability to manipulate an agent’s navigation in a simple maze environment with an intervention to a single channel in the network, providing the primary motivation for this work with the intent of extending these techniques to more complex multi-objective settings. The work specifically used an agent which had been trained in a modified version of the environment where the cheese was only ever added to the top right of the environment, and then deployed the agent into settings where the cheese could spawn at any location in the maze. This meant that there was a tension between the agent’s natural tendency to want to go to the top right of the environment and to go the cheese location.

They go on to show that the agent’s decision making about whether it travelled to the top right or towards the cheese alone was determined by both the Euclidean distance between the agent and the cheese and the path distance between the agent and the cheese. This means that the agent had multiple separate mechanisms driving its decisions rather than being a single unified policy. They did this by showing through regression analysis that if the agent was an equal path distance from two instances of cheese, it would tend to approach the cheese that was also closer in Euclidean distance.

This shows a surprising contradiction in the agent because visual shouldn’t meaningfully affect its policy and thus the agent must not be relying on a single reward maximising policy. A much better explanation is that the agent is operating under multiple competing goals.

They further demonstrate that neural networks can develop interpretable mechanisms to track goals. The authors identified specific channels that tracked the location of objectives in a maze environment and showed that these could be manipulated to control the agent’s behavior. They found that steering vectors, the difference in activations between otherwise identical states with and without an entity, could be used to influence the model’s behavior in predictable ways. This work serves as the foundation for our research, though we extend it to a multi-objective setting.

2.7.1 Discovering and Controlling Goal Representations

The authors successfully isolated the neural representation of the ‘cheese’ goal and demonstrated causal control over the agent’s behavior. First, they defined a ‘cheese vector’ by taking the difference in mean activations at a specific layer between trajectories where cheese was present and those where it was not:

$$v_{\text{cheese}}^l = \mathbb{E}_{s \sim \text{traj}_{\text{cheese}}} [\text{act}^l(s)] - \mathbb{E}_{s \sim \text{traj}_{\text{no-cheese}}} [\text{act}^l(s)]$$

Adding this vector back into the network’s forward pass, scaled by a coefficient α , allowed them to controllably amplify or suppress the agent’s cheese-seeking behavior.

Their most significant contribution was demonstrating this control causally through *activation patching*. They intervened on the final convolutional layer of the policy network by copying the activation pattern from a small spatial region of a channel corresponding to a real cheese location and ‘pasting’ it into the same channel at a different, empty location. By doing this they could reliably steer the agent to the empty location as if the cheese were there.

This effect was highly localized. They identified that out of 16 channels in the layer, just 11 were responsible for this behavior. Ablating these “cheese channels” destroyed the agent’s ability to find the cheese, while leaving other navigational capabilities intact. This proves that goal representations can be sufficiently modular and concentrated to allow for targeted manipulation, motivating the use of more advanced techniques like SAEs to further dissect these representations in a more complex, multi-objective setting.

2.8 Gaps in the Literature

Prior to the research detailed in the accompanying papers, several areas in RL interpretability remained open for exploration:

- Understanding how RL agents track and switch between multiple objectives, especially when those objectives are not the immediate next target.
- The application and efficacy of techniques like Sparse Autoencoders (SAEs) for disentangling representations specifically within RL agent architectures, particularly CNNs.

- Investigating how different aspects of neural network activation encode useful information for the next layer such as how activation strength within a channel might encode information like entity identity.
- The role and potential utility of ‘dead’ channels in SAEs for interpretability, which cease to activate during normal operation but might retain learned decoder weights.
- The generalization of interpretability findings from simpler RL agents to more complex architectures like decision transformers or RNN-based agents, especially for tasks requiring memory.

2.9 Summary

Prior work shows huge potential in the applications of RL agents due to their ability to achieve superhuman performance generalise broadly. However, the opacity of how this is achieved represents a persistent problem. A resurgence of interest in this area due to its application in frontier AI models adds urgency to better understand these models and how they operate on a mechanistic levelMini et al. [2023], Hilton et al. [2020]. Techniques such as activation analysis, steering vectors, and feature visualization have provided initial windows into these systems. However, challenges like polysemanticity (where neurons or channels respond to multiple unrelated conceptsOlah et al. [2020], Elhage et al. [2022]) and the distributed nature of representations often make it challenging to make strong conclusions about the operation of the neural network.

The application of tools like Sparse Autoencoders offers a potential solution for being able to make confident statements about whether the nature of how models truly learn to encode concepts[Elhage et al., 2022, Bricken et al., 2023], potentially making the functional roles of different network components clearer. While much of the early MI work focused on LLMs, applying and adapting these techniques to RL, with its emphasis on clear goals and environments with fewer components and potentially broadly generalisable solutions offers the ability to make unique insights that may apply to ML models more broadly.

The research presented in the accompanying papers (forming the basis of this thesis) builds directly on this existing work by:

- Extending activation steering and intervention techniques to more complex, multi-objective RL environments like Procgen Heist.

- Applying Sparse Autoencoders to the convolutional layers of an RL agent to disentangle features and investigate the extent to which multiple entities are truly captured within a single layer of the model and through what mechanism this is done.
- Systematically analyzing the role of individual channels in navigation and goal pursuit through methods like ablation studies and retargeting interventions.
- Exploring novel mechanisms of information encoding, such as multiplexing entity information within a single channel through varying activation strengths.

These contributions aim to deepen our understanding of how RL agents represent and pursue goals, and to refine the tools available for mechanistic interpretability for use in this domain.

Chapter 3

Methodology

3.1 Environment and Model

3.1.1 The Procgen Heist Environment

The Procgen Heist environment, from the suite of procedurally generated environments by [?](#) , was chosen for this research. The agent’s objective is to navigate a maze to collect a gem, which may require first collecting up to three keys (in the fixed order of blue, green, red) to open corresponding locks. The environment’s procedural generation varies the maze layout and the number of key-lock pairs present in each episode, from zero to three.

This environment provides a sparse reward signal, with a reward of +10 for reaching the gem and 0 otherwise. This design, coupled with the variable difficulty, establishes a natural curriculum that allows agents to succeed with random exploration in simpler levels before developing a more sophisticated policy.

The choice of a Procgen environment is motivated by the need to learn generalizable representations. The procedural nature of the mazes discourages the agent from memorizing simple heuristics and instead promotes the development of robust internal models of entities and game mechanics. Furthermore, Heist’s multi-objective nature is expected to produce complex internal behaviors, such as general-purpose navigation circuits that can be dynamically retargeted based on the current goal.

For this investigation, the ‘easy’ distribution of the environment was used, which provides the full 64x64 observation to the agent, making the environment a fully-observable Markov Decision Process (MDP). While memory-

based, partially-observable variants exist, they were not used. The distribution of entities is noteworthy: since levels are sampled uniformly from all configurations, the gem and the blue key/lock appear more frequently than the green and red pairs.

3.1.2 Model Architecture

Following the precedent set by ? for creating more interpretable models, this research primarily utilized a simplified version of the standard IMPALA architecture [?]. The model consists of four convolutional blocks followed by a fully connected layer. Each convolutional block contains a convolution operation and a pooling layer. This design features fewer convolutional layers and more dense layers than the original IMPALA network, a trade-off made to enhance interpretability without a significant loss in performance. While key findings were validated on a standard IMPALA-based architecture to ensure generality, the results presented in this thesis predominantly use this simplified architecture.

3.1.3 Model Training

The policy models were trained using Proximal Policy Optimization (PPO) with an open-source implementation based on PFRL, specifically designed for Progen environments. Training was conducted until convergence, which in this case was when no statistical improvement was observed for 10000 update steps. The primary model was trained for approximately 600 million steps.

To explore the effects of training dynamics on the final learned representations, experiments were run with different training configurations. These variations primarily involved altering the batch geometry through four main hyperparameters: the number of parallel environments, the number of steps per rollout, the mini-batch size, and the number of epochs.

In the primary configuration reported in this thesis, we used a setup of ($N_{\text{env}} = 64$, $N_{\text{step}} = 256$, `batch_size` = 8, $n_{\text{epoch}} = 3$). In this geometry, the rollout buffer contains $N_{\text{env}}N_{\text{step}} = 16\,384$ transitions. This is reshaped into 8 mini-batches, each containing $B_{\text{mb}} = 2\,048$ samples. The optimizer performs $8 \times 3 = 24$ gradient steps per PPO update, processing a total of $16\,384 \times 3 = 49\,152$ sample-updates per cycle. This configuration was chosen to balance stable learning from large batches with reasonable training time per update.

We ran 5 sets of training runs for this geometry in order to confirm the repeatability of our results, and ran further training runs with other geometries to observe the extent to which this altered the final structure of the model weights.

3.2 Interpretability Techniques Employed

3.2.1 Sparse Autoencoder (SAE) Training

SAEs were trained on the CNN layers of the Procgen model. For each SAE, recovery of 95% of the performance within the environment was often achieved within only a few thousand training steps, with variance explained approaching 1 in a similar time frame. Models were trained for 6 million steps (later stated as 15 million steps in Paper 1, clarification may be needed) to allow for convergence. An SAE was also trained on conv4a with a reduced number of channels (20, down from 32) to assess performance under compression. Techniques from Conerly et al. [2024] such as scaling the feature activations of the SAE by the norm of their corresponding decoder columns were used. This provides greater flexibility to the SAEs during training while preserving desired sparsity properties, helping to lower loss and reduce dead features. Recent work from ? on training SAEs on CNN layers provided essential groundwork. Similar hyperparameter choices for different layer depths were used, along with their approach of oversampling the strongest activations to avoid wasting computation. As in ?, 1x1 convolutional layers were used, generally applying an expansion factor of 4. To ensure stable training, activation normalisation and a warmup on the L1 coefficient were applied. Table ?? (from Paper 1) summarizes expansion factors and L1 coefficients.

Table 3.1: Expansion factors and L1 Coefficients for the different SAEs (from Paper 1)

Layer	Expansion Factor	L1 Coefficient
conv1a	2	1e-4
conv2a	4	5e-4
conv2b	4	5e-4
conv3a	4	5e-4
conv4a	4	5e-3

3.2.2 Feature Visualization

Two main approaches to feature visualization were used: synthetic visualization and finding maximally activating samples. Synthetic visualization involves using gradient descent to iteratively adjust an input image with the optimization target of getting the strongest possible average activation across a channel in a target channel, layer, or single neuron (focus was on whole channels). The strength of activations across the width and height of a given channel was summed, and the mean activation multiplied by -1 was used as the loss to minimize. Techniques from Olah et al. [2017] were used to improve synthetic visualizations, such as applying jitter, rotations, and de-correlating visualizations from within the color spectrum during gradient descent, then re-applying colors using a matrix developed from environment samples. Max activation sampling involves selecting samples from a dataset that produce the highest mean activation score across a given layer. To avoid duplicates, a diversity score (cosine similarity penalty) was applied. This technique focused on specific patches within the image rather than the whole image.

3.2.3 Steering Vectors and Activation Interventions

Steering Vector Calculation

As demonstrated in ?, objective steering vectors are computed. For a given maze m , layer l , and objective k at position x_k , with all objectives $\sum_i x_i$ and agent starting position $x_{\text{agent}}^{\text{start}}$, the steering vector is:

$$\text{SteeringVector}_k^l(m, x_k) = \text{Activ}^l(m, \sum_i x_i, x_{\text{agent}}^{\text{start}}) - \text{Activ}^l(m, (\sum_i x_i) - x_k, x_{\text{agent}}^{\text{start}})$$

An intervention involves modifying activations:

$$\text{NewActiv}^l(m, \sum_i x_i, x_{\text{agent}}) = \text{Activ}^l(m, \sum_i x_i, x_{\text{agent}}) + \alpha \cdot \text{SteeringVector}_k^l(m, x_k)$$

where α is an intervention coefficient. A negative α steers the model away from objective k .

Activation Manipulation and Patching

For scaled-up steering experiments, a specific entity was placed into the environment. The network was intervened upon by setting all its values to 0 and setting a specific region of a channel to a specified value. A parameter sweep over a range of values (typically 0 to 3) was performed. Episode lengths were

kept short. This approach was scaled by running a static intervention across every channel in the SAE, measuring deviation to the targeted area. For each entity-channel combination, 100 trials were run with activation value incrementing: $\text{activation value} = \text{trial number} \times (\text{max range} / \text{number of trials})$. A box maze (see Figure ?? in Paper 1) was used to provide a clear decision point.

For direct model control using activation patching, an activation vector from the environment for a given entity and layer was extracted. A specific channel’s activation values were edited to indicate an entity should be in a specific position, and this patched steering vector was added back into the model’s activations. Over 1000 activation patching experiments were performed across different channels and layers, with intervention coefficients from -2.0 to 2.0 in increments of 0.1, each with 100 episodes.

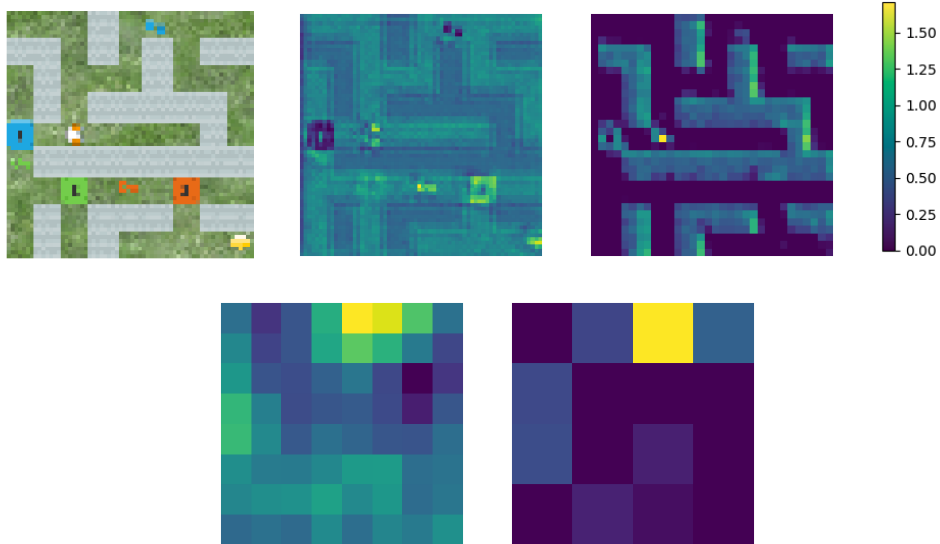


Figure 3.1: Visualization of the maze and neural network activations in different layers. Top row (left to right): observation input, layer 1 channel 10, layer 2 channel 10, and activation colorbar. Bottom row: layer 8 channel 29 and layer 9 channel 29. In the top row filters we observe a minimal activation around the regions of the blue key and lock and in the bottom row we see maximal activations around the blue key. This is a replication of the approach used by Mini et al. [2023] in order to get an initial sense of the clarity with which entities could be seen in activations.

3.2.4 Ablation Studies

To systematically assess the role of individual channels, a comprehensive ablation methodology was developed. For each channel in the SAE, episodes were run with only that channel active while ablating all other channels. This allowed measurement of the isolated capability of each channel to support navigation to different entities in the environment. Successful entity collection counts were recorded for each channel across multiple episodes.

3.3 Experimental Design and Evaluation Metrics

3.3.1 General Experimental Approach

The research involved training RL agents and corresponding SAEs, followed by a series of analytical experiments. These experiments included feature visualization to understand learned representations, activation steering and patching to test causal control over agent behavior, and ablation studies to determine the functional importance of individual channels. The primary goal was to dissect how the agent represents and pursues multiple objectives in the Procgen Heist environment and how SAEs might alter or clarify these representations.

3.3.2 Evaluation Metrics

Several metrics were used to evaluate the different components of the research:

- **For SAE Training (Paper 1):** Performance was primarily tracked using variance explained (aiming for close to 1) and the reward attained on the maze-solving task after SAE integration (>95% recovery). Sparsity was monitored by examining the number of dead features, the mean number of activating neurons, and the fraction of active neurons (target 0.001).
- **For Steering and Activation Interventions (Paper 1 & 2):** Success was measured by the agent’s deviation to the artificially targeted zone or behavior change (e.g., moving away from an objective). For scaled-up experiments (Paper 1), the frequency of successful deviation across channels and activation strengths was recorded. For activation patching (Paper 2), the ability to steer the agent predictably was the

primary metric.

- **For Ablation Studies (Paper 1):** The key metric was the count of successful entity collections per channel when all other channels were ablated, providing a quantitative measure of each channel’s specialization and capability.
- **For Steering Vector Analysis (Paper 2):** The effectiveness of steering vectors was partly assessed by the steerability of different layers (reduction in objective pickups). Channel function was investigated by measuring the frequency with which a channel in an objective steering vector activated most strongly at the location of that particular objective.

3.4 Summary

This chapter detailed the methodologies employed to investigate goal representation and control in a deep reinforcement learning agent. It began with a description of the Progen Heist environment and the model architectures used. Key interpretability techniques were then outlined, including the training of Sparse Autoencoders, feature visualization methods, the calculation and application of steering vectors for activation interventions, and systematic ablation studies. Finally, the experimental design and the quantitative and qualitative metrics used for evaluation were presented, providing a framework for understanding the results discussed in the subsequent chapter.

Chapter 4

Results

4.1 SAE Training and Feature Visualization

4.1.1 SAE Performance

For each Sparse Autoencoder (SAE) trained on the CNN layers of the Progen model, it was found that performance on the original task could be largely recovered. Specifically, the SAEs enabled the model to recover $\geq 95\%$ of its original performance within the Progen Heist environment (the conclusion of Paper 1 mentions $\geq 98\%$ performance recovery). This level of performance was often achieved within only a few thousand training steps for the SAEs. Concurrently, the variance explained by the SAEs approached 1 (indicating near-perfect reconstruction of the original activations) in a similar timeframe. The SAEs were typically trained for 6 to 15 million steps to ensure convergence.

An experiment was also conducted where an SAE was trained on the ‘conv4a’ layer with a reduced number of channels (20, down from the original 32 for that SAE). This compressed SAE was still able to recover most of the model’s performance, although to a lesser extent than the SAEs with higher expansion factors.

4.1.2 SAE Feature Visualization Results

The application of SAEs provided significantly more clear feature visualizations compared to those derived from the original base model, where visualizations often consisted primarily of noise.

While creating useful interpretations from synthetic visualizations (e.g., by optimizing input images to maximize channel activation) proved almost impossible, the method of finding maximally activating regions or patches from the dataset provided surprisingly positive and interpretable results (see Figure ?? for an example from Paper 1). This observation confirms similar findings reported by ?.

4.2 Steering and Activation Intervention Results

4.2.1 General Steering Capabilities

Activation steering interventions were successful in multiple layers, notably conv3a and conv4a, for both the SAE-enhanced models and the base model (Paper 1). The conv3a layer was identified as particularly steerable, especially when an SAE was applied to it, partly attributed to the increased number of channels in the SAE’s hidden layer (Paper 1).

In the SAE-enhanced model, channels in sae-conv4a appeared to represent identity-aware features whose influence on navigation was conditional on the agent’s policy, offering a more targeted control mechanism than the more general channels found in the base model (Paper 1).

Investigations with the base model (Paper 2) revealed that the green key was highly steerable. There was a notable improvement in the steerability of all objectives at layer 8 of the simplified architecture (equivalent to conv4b), followed by a sharp decrease in steerability in the subsequent pooling layer (layer 9). This pattern suggested that layer 8 might act as a crucial interface for processing salient information before it is passed to the fully connected layers for action selection (Paper 2).

4.2.2 Quantitative Steering Results and Channel Analysis

Experiments demonstrated that the model is steerable across many different channels. This effect was particularly pronounced in the conv3a layer. This also indicated that even after SAE training, a high degree of polysemanticity can persist in some layers, such as conv3a. However, the number of polysemantic channels significantly dropped in conv4a after SAE application. This suggests that signals related to targeting are shaped in conv3a and then re-

fined into a more pointed representation in conv4a (Paper 1, see Figure reffig:steering-dot-plot for entity distribution by channel and activation).

Analysis of steering vectors in the base model (Paper 2) showed them to be highly sparse, meaning most entries are approximately zero, containing information primarily related to the objective of interest. By examining the indices of the strongest positive and negative activations within these steering vectors against the location of the objective, specific channels were identified. For instance, in layer 8, channel 29 was found to fire strongly (positive activation) at the location of the objective, while channel 4 activated negatively. These channels tended to respond more to objectives encountered earlier in the game’s sequence (e.g., blue key, then green key, then red key, then gem) (Paper 2, see Figure ??).

4.2.3 Specific Steering Phenomena

Entity Prioritization in Steering

An important observation regarding steering was the model’s adherence to the game’s sequential objectives. If an attempt was made to retarget the agent towards a later objective in the sequence (e.g., intervening on channel 22, associated with the green key) while an earlier, prerequisite objective (e.g., the blue key) was still present in the environment, the agent would ignore the intervention and prioritize the blue key. The reverse, however, was not true; successfully collected earlier keys did not prevent targeting of subsequent ones (Paper 1).

Role of Specific Channels

Detailed experiments revealed distinct roles for certain channels. For example, in one SAE, interventions on channel 110 could steer the agent, but interventions on channel 115 alone could not; however, combining interventions on channels 115 and 71 (while 110 was not manipulated) did result in steering. Channel 67 in sae-conv3a appeared to be generally active across different keys, suggesting a degree of generality. Channel 6 in the same layer also showed strong activation in multiple key-related steering scenarios, suggesting it might track keys in general. In the case of the red key (the final key), channel 6 could be the sole strongly activating channel, as no other keys remained (Paper 1).

4.2.4 Steering with Dead Channels (SAE)

One of the most surprising findings from the SAE intervention experiments was that the majority of channels effective for steering the agent were, in fact, "dead channels." These are channels that exhibited no activation under normal operating conditions of the model. It was confirmed that these channels remained dead regardless of the input, indicating they were not simply highly selective for rare features (Paper 1).

4.3 Channel Ablation Study Results (SAE)

Comprehensive ablation studies on the SAEs revealed a remarkable concentration of functionality in specific channels. By activating only a single SAE channel at a time (ablating all others), it was found that only a handful of channels could independently support the agent's navigation to different entities in the environment (see Figure reffig:ablation-heatmap from Paper 1 for a heatmap of these results).

Notably, channels 44 and 75, which were identified as non-dead channels capable of steering the agent through interventions, showed a weaker ability than some other channels to successfully navigate to all entities when in isolation during ablation. This implies some specialization in roles: channels like 13, for example, were very effective at guiding the agent to targets during ablation (suggesting they are good at identifying target locations and paths) but were not effective for steering via direct intervention (implying they do not unilaterally dictate the agent's goal focus when other signals are present).

The ablation results also revealed entity specialization patterns. Some channels performed better for earlier entities in the collection sequence (e.g., the blue key) than for later ones (e.g., the red key). This reflects the sequential nature of the task, where the agent must collect earlier keys before later ones become relevant (Paper 1).

4.4 Activation-Level Entity Encoding (SAE)

A key finding from the SAE analysis was that certain critical steering channels, specifically channels 44 and 75 in an SAE, encoded different entities at different activation strengths. This indicates a sophisticated mechanism for multiplexing information within a single channel, rather than requiring separate dedicated channels for each entity type.

By examining the distribution of successful interventions for different entity types (see Figure ?? from Paper 1), it was observed that these interventions occurred at distinct activation values within the same channel. For example, in channel 44, successful steering toward the blue key typically required lower activation values (around 0.5-0.7 in the paper’s example, though Figure 4 suggests 1.4-1.8 for blue key vs 0.5-1.0 for gem), while steering toward the red key needed higher values (e.g., 1.5-2.0). This activation-level encoding mechanism allows a single channel to represent multiple entity types, enabling efficient information encoding within a minimal set of active channels (Paper 1).

4.5 Polysemanticity Observations (Base Model)

In the Procgen Heist environment, analysis of the base model (without SAEs) revealed that channels in a given layer tended to attend to many different entities, exhibiting less specialization than might be found in simpler environments (such as the Maze environment studied by ?). This phenomenon is known as polysemanticity [?], where a single channel activates in response to multiple, often unrelated, features or concepts.

A heatmap illustrating the frequency with which channels in layer 8 (of the simplified architecture in Paper 2) maximally activated on the position of specific entities (lock, gem, player, key) showed this polysemantic behavior. While some channels exhibited fairly clear specialization (e.g., channel 32 activating on the gem in the example), many channels activated across a range of different entities rather than exclusively tracking a single objective (Paper 2, see Figure ??).

4.6 Summary of Key Findings

The experimental results provide several key insights into the workings of the RL agent and the effects of SAEs. SAEs were successfully trained to reconstruct activations with high fidelity and minimal impact on task performance, while offering clearer feature visualizations, particularly through max activating patches. Both base models and SAE-enhanced models were steerable via activation interventions in convolutional layers, with SAEs sometimes offering more targeted control and revealing layer-specific roles in goal processing.

Quantitative analysis highlighted the persistence of polysemanticity in some

layers even after SAE training, though SAEs could also reduce it in others. A surprising finding was the steerability afforded by dead channels in SAEs. Ablation studies underscored the concentration of navigational capabilities into a few key SAE channels, which also showed specialization for different entities or stages of the task. Critically, some SAE channels were found to encode multiple entities within a single channel by utilizing different activation strengths—a sophisticated form of information multiplexing. In contrast, the base model exhibited more widespread polysemanticity across its channels. These findings collectively advance our understanding of how RL agents represent and pursue goals, and how tools like SAEs can help to uncover these mechanisms.

Chapter 5

Discussion

5.1 Interpretation of Results

5.1.1 The Effectiveness of SAEs for Interpretability

The results suggest that while Sparse Autoencoders (SAEs) do aid in disentangling features within RL agents, there are nuances to their utility. In some respects, the base model, despite its polysemanticity, may offer more straightforward interpretations for certain phenomena due to clearer delineations between primary tracking channels and others before SAE processing. However, the emergence of unique, more disentangled channels for specific entities (like keys) within the SAE is a notable advantage for targeted analysis and intervention. The variability in results depending on SAE training duration also highlights that stability in SAE training is an important factor to consider in interpretability work.

5.1.2 The Dead Channel Phenomenon in SAEs

The finding that a majority of steerable channels in the SAEs were "dead channels" (i.e., channels that exhibit no activation under normal circumstances) is a significant observation. This seemingly counterintuitive result has a clear mechanistic explanation rooted in the SAE architecture used (1x1 convolutional layers). In such an architecture, there is a direct one-to-one relationship between hidden channels in the SAE and their corresponding decoder weights. When a hidden channel dies (ceases to activate during normal forward passes), the gradient flow to its associated decoder weights stops. This effectively freezes those decoder weights at the values they had achieved

before the channel became inactive. Crucially, even though these channels do not activate during normal operation, their frozen decoder weights can still respond to artificial activation during intervention experiments. This explains why steering the agent is possible using channels that are otherwise silent – the learned patterns in their decoder weights are preserved and can be externally triggered. This implies that dead channels should not be automatically dismissed as unimportant in SAE analysis; they may preserve earlier learned representations that offer valuable insights into the model’s learning trajectory and latent capabilities.

5.1.3 Activation-Level Entity Encoding in SAEs

The discovery that specific SAE channels (e.g., channels 44 and 75 in Paper 1) encode multiple distinct entities by utilizing different activation strength ranges reveals a sophisticated information compression mechanism. Instead of requiring separate, dedicated channels for each entity type, the SAE learns to use activation magnitude as an additional dimension for encoding information. This has important implications for intervention strategies, suggesting that merely identifying *which* channels to modify might be insufficient; the *precise activation values* used for intervention are also critical. This activation-level encoding helps explain how the SAE can maintain high functional performance with a surprisingly small number of concurrently active channels—the network effectively multiplexes information within individual channels. This represents a form of representational efficiency that might not have been anticipated. An explanation for this phenomenon could be that due to the similarity of the task of tracking each key, it proves to be an efficient solution for the SAE to dedicate very few channels to this task and have them signal the specific target entity sequentially via activation strength. It remains an open question whether this functionality observed in the SAE reflects a similar underlying mechanism in the base model.

5.1.4 Role Specialisation within the SAE

A surprising finding was the differentiation in channel roles within the SAE. While certain channels (e.g., 44 and 75) demonstrated the ability to unilaterally alter the agent’s navigation through direct intervention (steering), many other channels were more reliably able to guide the agent during ablation studies (when they were the only active channel) but were completely unable to individually steer the agent against other prevailing signals. This indicates a specialization where the function of unequivocally dictating “go to this spot” was exclusive to a few channels like 44 and 75. Other channels,

more successful in ablation-based navigation, likely had a role in identifying *how* to get to a position; in the absence of conflicting signals (as in an ablation setting), this navigational information would lead the agent to the target, but these channels lacked the decisive influence to override other goal signals in a non-ablation context.

5.1.5 Polysemanticity and Goal Representation in the Base Model

While prior work demonstrated interpretable channels in simpler environments (e.g., Mini et al. [2023] in Procgen Maze), agents trained in more complex, multi-objective environments like Procgen Heist tend to develop highly polysemantic representations in their base networks. Most early-layer channels, and many in later convolutional layers, activate across multiple, diverse entities, making it challenging to assign singular, clear functional roles. Despite this polysemanticity, steering vectors proved to be a useful tool for isolating goal-related activity. Measuring the sparsity of these vectors allowed for the identification of channels in later layers that reliably track specific goals, even when these goals appear in varied spatial configurations. These results suggest different layers of abstraction within the model: lower layers appear to capture general visual features and multiple entity types, while later layers consolidate this information into more actionable, goal-directed signals. The methods used offer a diagnostic approach to probing this transition and identifying functionally important parts of the network, even amidst widespread polysemanticity.

5.2 Theoretical Implications

The findings from this research contribute to the theoretical understanding of mechanistic interpretability in deep reinforcement learning in several ways. The demonstration of activation-level encoding within SAE channels suggests that neural networks may employ more nuanced information encoding strategies than simple feature presence/absence, utilizing the analog nature of activations as an additional information dimension. This challenges some simpler models of feature representation and highlights the potential for information multiplexing as a core principle of neural computation, especially when under pressure for representational efficiency.

The dead channel phenomenon in SAEs, and its mechanistic explanation, provides a concrete example of how network components that appear non-

functional during standard inference can retain latent capabilities and learned knowledge. This has implications for how we assess model capacity and robustness, and for understanding the learning dynamics that lead to such states.

Furthermore, the observed specialization of roles within SAE channels (e.g., steering-dominant vs. navigation-supportive channels) hints at a distributed yet specialized computational architecture emerging from unsupervised SAE training. This could inform theories on how complex behaviors are decomposed and represented within neural networks.

For RL interpretability specifically, this work underscores that techniques successful in one domain (e.g., LLMs) or simpler environments may need adaptation or re-evaluation in more complex, interactive settings. The interplay between learned environmental structure (like sequential objectives in Heist) and emergent network representations is a key area for future theoretical development.

5.3 Practical Applications

While this research is primarily foundational, aiming to understand the internal workings of RL agents, the insights gained could have several long-term practical applications. A deeper understanding of how agents represent goals and make decisions can inform the development of more robust and reliable RL systems. For instance, identifying critical control channels or specific activation patterns linked to desired (or undesired) behaviors could lead to methods for:

- **Enhanced Safety and Control:** Modifying or monitoring these identified neural correlates could provide mechanisms to prevent catastrophic failures or to guide agent behavior in critical situations.
- **Improved Debugging and Diagnostics:** When an RL agent fails or behaves unexpectedly, the ability to inspect specific features or channels (especially those disentangled by SAEs) could significantly speed up the debugging process.
- **More Sample-Efficient Learning:** Understanding what representations are crucial for a task might eventually lead to techniques that encourage or bootstrap the learning of these representations, potentially improving sample efficiency.
- **Agent Alignment:** Identifying how an agent represents complex or

potentially conflicting goals is a step towards ensuring that agents behave in accordance with human intentions, a key challenge in AI alignment.

The techniques themselves, such as targeted activation steering or the analysis of SAE features, could become part of a standard toolkit for RL practitioners seeking to verify and validate their models beyond simple performance metrics.

5.4 Limitations of the Study

This research, while providing several insights, is subject to certain limitations:

- **Specificity to Architecture and Environment:** The findings are based on specific CNN architectures trained on the Procgen Heist environment. While efforts were made to use simplified, interpretable models and to validate on standard architectures (Paper 2), the extent to which these detailed mechanistic findings generalize to vastly different architectures (e.g., Transformers, RNNs) or other complex RL environments remains an open question.
- **Entangled Activations:** As noted in Paper 2, many goal-related activations in the base model remain entangled across channels. While SAEs aim to disentangle these, the disentanglement may not be perfect, and some interpretations might still be confounded by residual polysemanticity or distributed representations.
- **Off-Distribution Behavior:** Intervention experiments, by their nature, can push the network into off-distribution activation states. While this is useful for causal testing, the agent’s behavior under such strong artificial interventions might not fully reflect its operational characteristics under normal, on-distribution conditions.
- **Incomplete Mapping of Channel Functions:** While specific channels and their roles were identified, a complete, precise map of all channels and their exact contributions to various sub-tasks or representations was not achieved. The complexity of the network, even simplified ones, makes such exhaustive mapping a significant challenge.
- **SAE Interpretation Nuances:** The interpretation of SAE features themselves can be complex. It is not always clear if the features learned by an SAE perfectly mirror the computations of the base model or

if they represent a re-factorization that is useful but distinct. The stability of SAE training and its impact on the consistency of learned features across different training runs or minor hyperparameter changes is also a consideration (Paper 1).

- **Focus on Convolutional Layers:** The primary focus of intervention and SAE application was on the convolutional layers. While these are crucial for visual processing and early planning, a full understanding would also require similar deep dives into the fully connected layers and the policy/value heads.

5.5 Future Research Directions

The findings and limitations of this work point to several promising avenues for future research:

- **Tracking Non-Current Objectives:** Further exploration is needed into how RL agents internally represent and track entities or sub-goals that are not the immediate next target in a sequence, and whether interventions can reliably switch the agent’s focus to these deferred objectives.
- **Application to Different Architectures:** Extending this line of inquiry to more complex agent architectures, such as decision transformers or RNN-style networks, would be valuable. These architectures have different mechanisms for handling temporal information and memory, and training probes or performing targeted interventions in their residual streams or hidden states could yield new insights.
- **Deeper Investigation of Activation Strength Encoding:** The phenomenon of activation strength encoding entity identity within SAE channels warrants further investigation. Understanding the precise mechanisms and robustness of this encoding could be highly beneficial.
- **Exploring Other Environment Entities:** This work focused primarily on keys and the gem. Future studies could investigate if other visual cues (e.g., the image of collected keys in a UI element) trigger steerable behaviors related to corresponding objectives (e.g., locks).
- **Advanced Attribution Techniques:** Applying more advanced techniques like attribution-based parameter decomposition [Braun et al., 2025] to networks like the one studied here could yield valuable and

potentially generalizable insights into how parameters contribute to function.

- **SAEs and Base Model Correspondence:** Investigating whether the functionalities highlighted or preserved by SAEs (such as entity encoding via activation strength) truly reflect similar underlying mechanisms in the base model, or are emergent properties of the SAE compression process, is a critical follow-up question.
- **Robust Isolation of Goal-Related Circuits:** While steering vectors and SAEs help, further development of tools like causal tracing or more advanced sparse coding methods could be effective in more robustly isolating complete goal-related circuits, especially in the presence of significant polysematicity.
- **Impact of Environment Design on Learned Representations:** Investigating how variations in environment design (e.g., training models with no fixed ordering of objectives in the maze) affect the development of generalizable circuits for goal pursuit would be an interesting direction.
- **Synthesizing Distributed Information:** Developing tools or models capable of synthesizing information from across different parts of the network is crucial, as the factors contributing to a specific outcome are often distributed. SAEs are one step in this direction, but further methods are needed.

5.6 Reflections on the Research Process

(This section is for your personal reflections on the research journey, challenges faced, and lessons learned. This content should be written by you.)

Chapter 6

Conclusion

6.1 Summary of the Research

This thesis investigated the internal representations and goal-directed behavior of deep reinforcement learning agents, specifically focusing on a model trained on the multi-objective Procgen Heist environment. The research employed mechanistic interpretability techniques, including the training and analysis of Sparse Autoencoders (SAEs) on the agent’s convolutional layers, feature visualization, activation steering, and channel ablation studies. The primary aim was to understand how such agents represent complex, sequential goals and how tools like SAEs can assist in disentangling and interpreting these representations. Building on previous work that demonstrated control of RL agents via activation manipulation, this research extended these methods to a more complex setting and explored the utility of SAEs in revealing finer-grained details of neural computation within RL agents.

6.2 Key Findings and Contributions

The principal findings and contributions of this research are as follows:

- Successfully trained Sparse Autoencoders (SAEs) on the CNN layers of the Heist model, recovering more than 98% of the original task performance.
- Demonstrated that precise steering of the agent towards different entities could be achieved by intervening on single channels in multiple layers (‘conv3a’, ‘conv4a’) of both the base model and SAE-enhanced

models.

- Revealed that while some channels appeared to track specific entities and also affect steering, other SAE channels were highly efficient at tracking/navigation during ablations but could not unilaterally modify the agent’s trajectory, indicating specialized roles within the SAE.
- Uncovered the surprising property that a large number of ”dead channels” in the SAEs (channels with no activation during normal operation) could nonetheless be used to successfully steer the agent. This was explained as an artifact of the SAE’s decoder architecture retaining learned weights.
- Found that the model was still capable of navigating to different keys even when only a single SAE channel was active, highlighting a high degree of functional compression.
- Discovered that certain critical steering channels within the SAEs did not simply represent single entities, but rather encoded different entities at different activation strengths, revealing a sophisticated mechanism for multiplexing information within a single channel.

Additional key insights include:

- Confirmation that steering vectors can be a valuable tool for identifying goal-tracking channels and for applying targeted control, even in more complex, multi-objective environments like Procgen Heist.
- Observation that later convolutional layers in the base model (e.g., layer 8 in the simplified architecture) tend to exhibit more specialized behavior related to goal representation compared to earlier, more polysemantic layers.
- Reinforcement of the idea that simplified model architectures can facilitate interpretability in RL agents, as suggested by ?.

6.3 Implications for Theory and Practice

The results presented have several implications. Theoretically, they demonstrate that SAEs can offer more precise steering and control than might be possible with the base model alone, by disentangling general goal signals into more specific, identity-aware features. The influence of these features can be conditional on the agent’s policy, providing a targeted control mechanism.

Furthermore, this work illustrates how SAEs may compress specific functionalities into individual channels, and then encode further nuanced information (like entity identity) into the specific activation strength of those channels. This points towards more complex information processing strategies within neural networks than often assumed and has implications for understanding representational efficiency and the nature of learned features.

Practically, these findings may provide insight into the application of SAEs more broadly beyond language models, and help to shed light on the internal operations of reinforcement learning models. Understanding these mechanisms is a step towards building more reliable, predictable, and ultimately safer AI systems.

6.4 Recommendations

Based on the findings, practitioners applying SAEs to RL agents should be aware of the potential for phenomena like dead channel steerability and activation-level encoding, as these may hold important clues to the model’s function. Researchers are encouraged to further investigate the generalizability of these findings across different model architectures, RL environments, and SAE training configurations. A key recommendation stemming from this work is to not solely rely on coarse measures of channel activity (e.g., simple activation vs. no activation) but to consider the possibility of information being encoded in the precise magnitude of activations, especially when using tools like SAEs that promote sparse and disentangled representations.

6.5 Final Remarks

This research contributes to the growing field of mechanistic interpretability in reinforcement learning by providing a detailed analysis of goal representation and control in a complex environment. The application of Sparse Autoencoders has revealed nuanced ways in which information can be encoded and manipulated within an RL agent’s neural network. The phenomena observed, such as entity multiplexing through activation strength and the functional relevance of dead channels, open up new avenues for understanding and potentially controlling sophisticated AI systems. Ultimately, continued exploration of these internal mechanisms is crucial for advancing our ability to develop AI that is not only capable but also transparent and trustworthy.

Bibliography

- Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, November 2018.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016a.
- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016b.
- Akanksha Atrey, Kaleigh Clary, and David Jensen. Exploratory Not Explanatory: Counterfactual Analysis of Saliency Maps for Deep Reinforcement Learning, February 2020.
- Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheeraz El-Showk, Nelson Hatfield-Dodds, Tristan Hatfield-Dodds, Scott Hawkins, Tom Hume, Stanislav Krastev, Liane Lenain, Logan LHOLTS, David Israel, Tomasz Korbak, Guillaume Lample, Lauren Mantle, Kyla-bel Livingston, Catherine Manning, Michael McClanahan, Jared Quincy Olsson, Christopher Olah, Matheus Rahtz, Sorelle Ringer, Nicholas Sellitto, Scott Saunders, Ansh Sastry, Scott Seethor, Leslie Somogyi, Sarah Stair, Benjamin Stein-Perlman, Esin Lue, Sainbayar Sukhbaatar, Jason Tan, Benjamin Thompson, Helen Zen, Jared Kaplan, and Sam McCandlish. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022.
- Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

Jan Betley. Localizing goal misgeneralization in a maze-solving policy network. July 2023.

Dan Braun, Lucius Bushnaq, Stefan Heimersheim, Jake Mendel, and Lee Sharkey. Interpretability in Parameter Space: Minimizing Mechanistic Description Length with Attribution-based Parameter Decomposition, February 2025.

Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermy, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

Nick Cammarata, Gabriel Goh, Shan Carter, Ludwig Schubert, Michael Petrov, and Chris Olah. Curve detectors. *Distill*, 2020. doi: 10.23915/distill.00024.003.

Karl Cobbe, Oleg Klimov, Chris Hesse, Taehoon Kim, and John Schulman. Quantifying generalization in reinforcement learning. In Kamalika Chaudhuri and Ruslan Salakhutdinov, editors, *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 1282–1289. PMLR, June 2019.

Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging Procedural Generation to Benchmark Reinforcement Learning, July 2020.

Tom Conerly, Adly Templeton, Trenton Bricken, Jonathan Marcus, and Tom Henighan. Update on how we train SAEs, April 2024.

Fahim Dalvi, Nadir Durrani, Hassan Sajjad, and Preslav Nakov. On the effect of dropping layers of pre-trained transformer models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

J. Degraeve, F. Felici, J. Buchli, M. Neunert, B. Strouse, T. Hottiger, J. Byrnes, R. de Folter, A. Laterre, M. van de Walle, J. van Dorland, T. Laan, L. Ledsgaard, G. Sartoretti, K. Sünderhauf, A. Bridgwater, A. Ecuier-Wallaert, N. Heess, Y. Tassa, R. Hafner, M. Riedmiller, and

- T. Lillicrap. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897):414–419, February 2022. doi: 10.1038/s41586-021-04301-9.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. Toy models of superposition. *Transformer Circuits Thread*, September 2022.
- Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on PPO and TRPO. *arXiv preprint arXiv:2005.12729*, 2020.
- Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures, June 2018.
- Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(1):1437–1480, 2015.
- Liv Gorton. The Missing Curve Detectors of InceptionV1: Applying Sparse Autoencoders to InceptionV1 Early Vision, June 2024.
- Samuel Greydanus, Anurag Koul, Jonathan Dodge, and Alan Fern. Visualizing and Understanding Atari Agents. In *Proceedings of the 35th International Conference on Machine Learning*, pages 1792–1801. PMLR, July 2018.
- Wes Gurnee and Max Tegmark. Language models represent space and time. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

Jacob Hilton, Nick Cammarata, Shan Carter, Gabriel Goh, and Chris Olah. Understanding RL Vision. *Distill*, 5(11):e29, November 2020. ISSN 2476-0757. doi: 10.23915/distill.00029.

Evan Hubinger, Richard van der-Merwe, Vladimir Mikulik, Adam Lan, Sebastian Farquhar, Mark Whalen, John Bottomley, Lorand Nemeth, Zachary Kenton, Andrew Lohn, et al. Risks from learned optimization in advanced machine learning systems. *arXiv preprint arXiv:1906.01820*, 2019.

Gregory Kahn, Antonio Villaflor, Vitchyr Pong, Pieter Abbeel, and Sergey Levine. Uncertainty-aware reinforcement learning for collision avoidance. *arXiv preprint arXiv:1702.01182*, 2017.

Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas, and Rory Sayres. Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV), June 2018.

Victoria Krakovna. Specification gaming examples in AI.

Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Shan Carter, and Shane Legg. Faulty reward functions in the wild. *arXiv preprint arXiv:2001.00055*, 2020.

Lorenzo Langosco, Rohin Shah, Victoria Krakovna, Gillian Hadfield, Dario Amodei, et al. Goal misgeneralization in deep reinforcement learning. In *NeurIPS*, 2022.

Kenneth Li, Aspen K Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Emergent world representations: Exploring a sequence model trained on othello. *arXiv preprint arXiv:2210.13382*, 2022.

Thomas McGrath, Andrei Kapishnikov, Nenad Tomašev, Adam Pearce, Demis Hassabis, Been Kim, Ulrich Paquet, and Vladimir Kramnik. Acquisition of Chess Knowledge in AlphaZero. *Proceedings of the National Academy of Sciences*, 119(47):e2206625119, November 2022. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.2206625119.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and Editing Factual Associations in GPT, January 2023.

Ulisse Mini, Peli Grietzer, Mrinank Sharma, Austin Meek, Monte MacDiarmid, and Alexander Matt Turner. Understanding and Controlling a Maze-Solving Policy Network, October 2023.

Neel Nanda, Andrew Lee, and Martin Wattenberg. Emergent Linear Representations in World Models of Self-Supervised Sequence Models, September 2023.

Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. Feature visualization, November 2017.

Chris Olah, Arvind Satyanarayan, Ian Johnson, Shan Carter, Ludwig Schubert, Katherine Ye, and Alexander Mordvintsev. The building blocks of interpretability. *Distill*, 2018. doi: 10.23915/distill.00010.

Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom In: An Introduction to Circuits. *Distill*, 5(3):10.23915/distill.00024.001, March 2020. ISSN 2476-0757. doi: 10.23915/distill.00024.001.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Ashish Kumar Shakya, Gopinatha Pillai, and Sohom Chakrabarty. Reinforcement learning algorithms: A brief survey. *Expert Systems with Applications*, 231:120495, November 2023. ISSN 0957-4174. doi: 10.1016/j.eswa.2023.120495.

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016. doi: 10.1038/nature16961.

Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic Attribution for Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning*, pages 3319–3328. PMLR, July 2017.

Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT press, 2018.

Akshay Thatte, Anirudh G. Mini, Jacob Hilton, Aditi Khanna, Pieter Abbeel, and Rohan Taori Krishna. Insights into representations learned by reinforcement learning agents. In *NeurIPS 2022 Workshop on Visually Grounded Reinforcement Learning*, 2022.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Activation Addition: Steering Language Models Without Optimization, June 2024a.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering Language Models With Activation Engineering, October 2024b.